MIDTERM STUDY GUIDE

Operating Systems and System Programming · Chapters I–IV + Practice 1 + Labs (Weeks 1–7)

Condensed, comprehensive review for efficient midterm preparation

Program on disk	Process	Threads	Memory for the process	Pages and frames	File system persistence and organization

Scope note

This guide covers everything in the course up to Week 7: Chapter I (Introduction to OS and SP), Chapter II (Process and Thread Management), Practice 1 (Scenarios 1–6), Chapter III (Memory Management & Paging Lab), Chapter IV (File Systems & Directory Structures), and the Python labs. The midterm is worth **30** of the 100 course points.

Important exam note

The slides never state the midterm format. The only graded-style questions the instructor has shown are the **Practice 1 scenarios** (a situation, then "is this a process or a thread problem, and why?") and the **paging lab questions** (trace a page table, predict what the code does). Expect those two styles plus definitions and the comparison tables. Sections marked **Added** are not on the slides; they are there so a "trace it" question does not catch you unprepared.

Prepared from the chapter I–III slides, Practice 1, the paging assignment and my notes.

How to Use This Guide

This guide replaces re-reading four chapters, the practice sheet, and the labs. It keeps the chapter order and the instructor's wording, and removes repetition.

Study in three passes

Pass	What to do
1. Understand the story	The OS turns a program into a **process**, splits its work into **threads**, and gives it **memory** through pages and frames.
2. Review each chapter	Definitions and every comparison table. Most slides in this course **are** tables.
3. Test yourself	Do the six Practice 1 scenarios and the paging traces from memory, then the self-check section.

What "explain the scenario" means

- Decide whether the fact is **process-level** or **thread-level**.
- Justify with **address spaces**: separate (processes) or shared (threads).
- Name the mechanism: blocked UI thread, zombie, race condition, page table, IPC.
- Finish with the **fix** or the **trade-off**.

> **Do not memorize syntax**
> For the Python labs, understand what each call asks the OS for (`os.getpid()` = my process ID, `psutil.cpu_percent()` = CPU usage) and be able to trace the paging simulation. You do not need to write the programs from memory.

The Course as One System

Hardware	Kernel	System calls	Shell and utilities	Applications	User

Each part answers a different question:

Week	Chapter	Main question	Core concepts
1	Lab	How does a program ask the OS for things?	`os`, `platform`: OS info, listing files, creating a file, process ID
2	I. Introduction to OS and SP	What is an OS, and what is system programming?	OS roles, kernel / shell / file system, seven OS types, SP vs application programming, Python vs C
3	II. Process Management	What runs, and how?	Process, attributes, states, process types, thread, thread types, multiprocessing vs threading
4	Practice 1	Can you tell a process fact from a thread fact?	Frozen UI, zombies, Chrome tabs, process vs thread per request, race condition, shared vs private memory
5	III. Memory Management	Where does a process live?	Four goals, memory hierarchy, partitioning, paging, segmentation, virtual memory, page replacement, buddy, slab
7	IV. File Systems	How is data organized, stored, and protected persistently?	Files, attributes, operations, access methods (sequential, direct, indexed), directory structures (single-level, two-level, tree, acyclic graph, general graph), directory implementation (linear list, hash table, B-tree), layered file system structure, VFS, disk / network / special file systems (procfs, tmpfs), allocation methods (contiguous, linked, indexed), free space bitmaps, journaling, buffer cache, RAID 0/1/5, FS vs directory structure

Key distinction

Chapter II is about **execution and isolation** (processes and threads). Chapter III is about **volatile memory** (pages, frames, virtual memory). Chapter IV is about **non-volatile persistence** (files, directories, allocation methods, and data integrity across reboots).

Chapter I - What is an Operating System?

Definition

An **Operating System (OS)** is a software program that **manages computer hardware and software resources** and **provides common services for computer programs**. It acts as an **intermediary between users and the computer hardware**.

System software = programs designed to operate, control and extend the capabilities of the hardware (as opposed to application software, which solves a user's problem).

Five main roles

Role	What it does
Process management	Manages the execution of multiple processes
Memory management	Allocates and deallocates memory for programs
File system management	Handles file storage, retrieval and permissions
Device management	Controls and coordinates hardware components
User interface	Lets users interact with the system (GUI or CLI)

Three core components

Component	Description	Examples
Kernel	The core part of the OS; interacts **directly with the hardware**	Linux kernel, Windows NT
Shell	The user interface that takes commands and executes them	Bash, PowerShell
File system	Organizes and manages data storage	NTFS, FAT32

Seven types of OS

Type	Description	Examples
Single-user	One user at a time, limited multitasking	MS-DOS, Windows 95
Multi-user	Multiple users access the system simultaneously	Linux, Unix, Windows Server
Batch	Executes jobs in batches **without user interaction**	IBM OS
Real-time (RTOS)	Processes data in real time with **strict timing constraints**	VxWorks, QNX
Distributed	Manages networked computers **as a single system**	Amoeba, Plan 9
Embedded	For embedded systems with **limited resources**	Embedded Linux
Mobile	Optimized for smartphones and tablets	Android, iOS

Important

On the slide this table is printed **out of alignment** (for example "IBM OS" sits beside the real-time row). The pairing above is the correct one. If the exam reuses the slide, answer with the correct pairing.

Similarities and differences between OS types

Aspect	Similarity	Difference
Kernel	All have a kernel for hardware control	**Monolithic** (Linux) vs **microkernel** (QNX)
User interface	Most provide GUI and CLI	Some (servers) are CLI-only
Multitasking	Modern OS support multitasking	Real-time OS **prioritize task scheduling**
Security	All enforce user permissions	Multi-user OS have **stricter access control**

Monolithic kernel: most services (drivers, file systems) run inside the kernel. Fast, but a fault there affects everything. **Microkernel:** the kernel is minimal and services run as separate user-space processes. More robust, more communication overhead.

Chapter I - System Programming

Definition

System programming is the development of software that **provides core services to the computer hardware** and **enables the execution of application programs**. It means writing low-level code that interacts directly with the OS, hardware and system resources.

Four characteristics

Characteristic	Meaning
Close to hardware	Memory management, process control, device drivers
Interacts with the OS	Uses **system calls** to request services from the kernel
Performance-critical	Efficient management of CPU, memory and I/O
Low/medium-level languages	Mainly C, C++, Rust; sometimes Python for scripting

> **Key idea**
> A **system call** is how a program asks the kernel to do something it cannot do itself. The slide's examples are `fork()` (create a process), `exec()` (run a program) and `open()` (open a file).

System programming vs application programming

Aspect	System programming	Application programming
Level of abstraction	Low-level (direct hardware/OS interaction)	High-level (user-facing software)
Languages	C, C++, Assembly, Rust	Python, Java, JavaScript, C#
Purpose	Manages hardware and OS resources	Solves end-user problems
Examples	OS kernels, device drivers, shells	Web browsers, games, office software

Five key areas

Area	Examples
OS development	Kernels (Linux, Windows NT), system calls, memory management (paging, segmentation)
Device drivers	Software that lets the OS talk to printers, GPUs, USB devices; written in C or Rust
System utilities	Shells (Bash, PowerShell), debuggers (GDB), process monitors (htop)
Embedded systems	Firmware for microcontrollers (Arduino, Raspberry Pi), RTOS for robotics and cars
Networking and security	Sockets, firewalls, VPNs, authentication, encryption

Python vs C

Feature	Python	C
Ease of use	High-level, easy to learn and write	Low-level, complex memory management
Speed	Slower (interpreted)	Faster (compiled, direct hardware access)
Libraries	Rich ecosystem	Limited high-level abstractions
Portability	Cross-platform	Requires recompilation for each OS
Use case	Scripting, automation, rapid prototyping (`psutil` for process monitoring)	Kernel and driver development, real-time systems

In this course Python is used to **observe and script** the OS, not to build one.

Chapter II - Processes

Definition

A **process** is an **instance of a program in execution**. A program is a file on disk; a process is that program running, with state attached.

Program code (text section)	Current activity (program counter, registers)	Resources (memory, files, CPU state)

Characteristics

Property	Meaning
Isolation	Each process has its **own memory space**
Heavyweight	Requires more system resources
Context switching	**Slower**, because of memory isolation
Communication	Needs **inter-process communication (IPC)**

What a process has

Attribute	Meaning
Process ID (PID)	Unique numerical identifier
Program counter	The next instruction to execute
State	New, ready, running, waiting, terminated
Priority	Determines CPU access priority
I/O status	List of I/O devices allocated

The five states

New	Ready	Running	Waiting	Terminated

State	Meaning
New	Being created
Ready	Could run, but the CPU is busy
Running	On the CPU
Waiting	Needs something first, usually I/O
Terminated	Finished

Added: the transitions

The slide lists the states only. The moves between them are: new → ready (admitted), ready → running (the scheduler picks it), running → ready (its time is up or a higher-priority process arrives), running → waiting (it asks for I/O), **waiting → ready** (the I/O finishes; it does not jump straight back to running), running → terminated (it exits).

Seven types of process

Type	Description
Foreground	Interacts directly with the user
Background	Runs without user interaction
Daemon	Background service that **starts at boot**
Parent	Creates other processes
Child	Created by another process
Zombie	**Terminated**, but still has an entry in the process table
Orphan	A child whose **parent has terminated**

Do not confuse

A **zombie** is dead but not yet cleaned up (its parent has not collected the exit status). An **orphan** is still alive but has lost its parent, and the OS gives it a new one.

Chapter II - Threads

Definition

A **thread** is the **smallest unit of execution within a process**, sharing the process's memory and resources.

Property	Meaning
Shared memory	Threads in one process share memory
Lightweight	Less overhead than a process
Faster context switching	Because memory is shared
Simplified communication	**No IPC needed**

Three types of thread

Type	Managed by	Behavior	Slide example
User-level	User-level libraries	Fast, but **not visible to the OS**. If one blocks, the whole program can stall.	Game engine threads managing rendering
Kernel-level	The OS directly	Slower but powerful. If one blocks, **others continue**.	Web server handling multiple client requests
Hybrid	Both	Combines the two	Solaris

Processes vs threads

Feature	Process	Thread
Memory	Isolated	Shared
Creation overhead	High	Low
Communication	Requires IPC	Direct, via shared memory
Context switching	Slower	Faster
Resource usage	More	Less
Termination	Slower	Faster

Key idea
Every row comes from one fact: **processes are isolated, threads share**. Processes are safer; threads are cheaper.

Multiprocessing vs threading

	Multiprocessing	Threading
What it is	Multiple independent processes, each with its own memory	Multiple threads inside one process, sharing memory
Memory	Separate by default	Shared: all threads see the same globals
Weight	Heavyweight: slower to create and destroy	Lightweight
Execution	**True parallelism** on multiple CPU cores	**Concurrency**, not always parallelism
Best for	**CPU-bound** tasks (calculation)	**I/O-bound** tasks, responsive UIs, frequent communication

Concurrency = several tasks in progress, taking turns. **Parallelism** = tasks literally running at the same instant on different cores.

Practice 1 - Scenarios 1 to 3

These six scenarios are the instructor's own questions with model answers. Learn the reasoning, not the wording.

1. The frozen word processor

The app is unresponsive after opening a large file; the OS and other apps work.

Question	Model answer
Process-level or thread-level?	**Thread-level.** The process is alive (the OS still tracks it), but the **UI thread is blocked** on synchronous I/O or a long computation.
Why is the OS still responsive?	The OS is a separate set of processes in **different address spaces**, **scheduled independently**.
With multiple threads?	One thread for the UI and one for loading: the UI stays responsive during heavy work.

2. Entries with status Z

The process table fills with Z entries that use no CPU or memory, until no new process can be created.

Question	Model answer
What is it?	A **zombie process**: terminated, but the parent has not called **`wait()` or `waitpid()`** to collect its exit status.
Why does a dead process keep an entry?	The entry holds the **exit status** the parent must read. Until then the OS cannot free it.
What is the parent doing wrong? The fix?	It never calls `wait()`. Call `wait()` / `waitpid()` after forking, or install a **`SIGCHLD` handler** that reaps children.
Could threads do this?	**No.** A finished thread's resources are reclaimed by the process, and exit status is handled at process level. Joinable/detached is a different mechanism.

The real damage: the process table is **finite**. When it is full, no new process can start.

3. Chrome with 15 tabs

15+ `chrome.exe` processes; closing one tab kills exactly one process.

Question	Model answer
Why processes instead of threads?	**Isolation and security**: each tab has its own address space, so a crash or exploit in one tab cannot affect the others.
Primary advantage	**Fault isolation and security.** A bad page cannot read another tab's memory.
Primary disadvantage	**Resource overhead**: each process has its own address space, page tables, file descriptors and kernel structures; context switches cost more.
Why do the others survive a crash?	The OS tears down **only that process's memory**. The others have separate page tables.

Practice 1 - Scenarios 4 to 6

4. One process per request vs one thread per request

Design A (process per request) is more resilient. Design B (thread per request) handles more requests per second.

Question	Model answer
Why are thread creation and switching cheaper?	Threads share the address space, so the OS does not create or **switch page tables**, copy file descriptor tables, or set up a new address space. A thread switch swaps only **registers, stack pointer and program counter**. A process switch changes page tables and often **flushes the TLB**.
Why does A survive a crash that kills B?	In A the OS kills only the crashed process. In B all threads share one address space, so one bad pointer brings down **the entire process**.
Both performance and isolation?	A **hybrid**: a **process pool** where each process runs a **thread pool** (the model of modern web servers).

The slides skip part (c) of this question.

5. The lost increments

Two threads each do `counter++` one million times. Expected 2,000,000; actual result is lower and different every run.

Load counter into a register	Increment the register	Store it back

Question	Model answer
Why is the result too low?	`counter++` is **not atomic**. Two threads load the same value, both increment, both store: **one increment is lost**.
Name of the phenomenon	A **race condition**.
Whose fault: threads, scheduler or programmer?	The **programmer**. The OS offers **mutexes, semaphores, atomics**, but does not protect shared variables automatically.
Two processes with shared memory instead?	**The problem persists.** With private copies there is no race, but each process counts 1,000,000 on its own.

A result that **changes every run** is the signature of a race condition.

6. Globals are visible, locals are not

Thread B sees Thread A's change to a global but cannot see A's local variable.

Shared between threads	Private to each thread
Code segment	Stack
Data segment (globals)	Registers
Heap	Program counter
Open file descriptors	Thread-local storage (TLS)
Current working directory	
Signal handlers	

- **Why:** globals live in the process's **data segment** (shared); locals live on each thread's **own stack** (private).
- **If these were two processes:** the global is **not shared by default**. Sharing needs explicit IPC: shared memory, pipes, message queues.
- **Benefit and danger:** communication is fast (no kernel, no copying), but **any thread can corrupt shared data**, and the whole burden of synchronization falls on the programmer.

High-yield

The shared/private table answers many questions by itself. Remember it as: **everything is shared except what a thread needs to run on its own**: stack, registers, program counter, TLS.

Chapter III - Goals and Memory Hierarchy

Definition

Memory management is the process of **allocating, tracking and freeing memory** in a computer system to ensure efficient execution of programs.

Four goals

Goal	Meaning
Allocation	Provides memory to processes when needed
Protection	Prevents processes from accessing **unauthorized** memory
Sharing	Allows **controlled** memory sharing between processes
Optimization	**Minimize fragmentation**, maximize utilization

Memory hierarchy

The structured arrangement of memory types by **speed, cost and capacity**. It keeps the most frequently accessed data in the fastest memory.

Level	Speed	Size	Cost per bit	Example
CPU registers	Fastest	Very small	Very high	Temporary computation storage
Cache (L1, L2, L3)	Very fast	Small	High	Frequently used instructions
Main memory (RAM)	Moderate	Medium	Moderate	Active programs and data
Secondary storage	Slow	Large	Low	HDDs, SSDs
Tertiary storage	Slowest	Very large	Very low	Backup tapes, optical disks
Virtual memory	Slowest	Very large	Lowest	Swap space on disk

> **Important**
> The slide's version of this table is also printed with **shifted columns**. The order above follows the rule the slide itself states: faster memory is smaller and more expensive.

Going down the hierarchy: slower	bigger	cheaper per bit

Three characteristics

Characteristic	Meaning
Response time	Faster memory is placed **closer to the CPU**
Cost vs capacity	Faster memory is more expensive and smaller; slower memory is cheaper and larger
Locality of reference	Programs reuse the same data (**temporal** locality) or nearby data (**spatial** locality), which justifies keeping it in fast memory

Chapter III - Allocation Techniques

1. Contiguous allocation

Each process gets memory in **one continuous block**.

	Fixed partitioning	Dynamic partitioning
How	Memory divided into fixed-size partitions **at boot time**	Memory allocated **as needed** in variable-sized blocks
Rule	Each partition holds exactly one process	Each process gets exactly what it requires
Problem	**Internal fragmentation**; a large process may not fit	**External fragmentation**; needs periodic **compaction**

Do not confuse
Internal fragmentation = wasted space **inside** an allocated block (fixed partitioning, the last page in paging).
External fragmentation = wasted gaps **between** allocated blocks (dynamic partitioning, segmentation).

2. Non-contiguous allocation

Type	How	Advantages	Disadvantages
Paging	Physical memory in fixed-size **frames**, process in same-size **pages**, mapped by a **page table**	**No external fragmentation**; enables virtual memory	Small internal fragmentation (last page); page-table overhead
Segmentation	Variable-size **segments** matching program structure (code, data, stack); **segment table** with base and length	Matches the programmer's view; easy sharing (shared libraries)	**External fragmentation**; complex management
Segmented paging	Segments divided into pages	Benefits of both	Used in modern systems such as **x86**

Page = block of a process's **logical** memory. **Frame** = same-size block of **physical** memory. The **page table** maps page → frame.

3. Virtual memory

A mechanism using hardware and software where **part of secondary storage** gives the illusion of being **part of main memory**. It lets many applications run with little main memory and raises the **degree of multiprogramming**. It is commonly implemented with **demand paging**.

Technique	Meaning
Demand paging	Pages are loaded **only when needed**; a **page fault** triggers loading from disk
Page replacement	Chooses a page to remove when **no free frame** is available

Algorithm	Replaces	Note
FIFO	The **oldest** page	Simple, poor performance
LRU	The **least recently used** page	Near-optimal, expensive to implement
Optimal	The page **not needed for the longest time**	Needs the future: a theoretical benchmark

A **page fault is not an error**. It means "this page is not in memory yet", and the OS loads it.

4. Buddy system and 5. Slab allocation

	Buddy system	Slab allocation
Idea	Memory in **power-of-2** blocks	**Pre-allocated slabs** for frequently used kernel objects
Process	Find the smallest sufficient block; if none, **split** a larger block into two buddies; on free, **merge** adjacent free buddies	Objects are taken from and returned to a ready slab
Benefit	Efficient allocation	No fragmentation for small objects; fast allocation and deallocation
Used in	**Linux kernel**	**Kernel objects**

Comparison of techniques (slide table)

Technique	Contiguous?	Fragmentation	Overhead	Used for
Fixed partitioning	Yes	Internal	Low	Early OS
Dynamic partitioning	Yes	External	Medium	—
Paging	Yes / No	Small internal	Medium	Modern OS
Segmentation	Yes / No	External	High	Program memory
Buddy system	Yes / No	Minimal	Medium	Kernel memory
Slab allocation	No	None	Low	Kernel objects

Important

The "Yes / No" entries are how the slide prints them. Read paging as: the process **looks** contiguous in logical memory (pages 0, 1, 2, 3) but its frames are **not adjacent** in physical memory. If asked directly, paging and segmentation are **non-contiguous** techniques, which is how the slides classify them in the section heading.

Added - Worked Examples

None of these traces are on the slides. They are included because "apply the algorithm" is the natural exam question for this chapter.

Translating an address with a page table

Logical address	Page number and offset	Page table gives the frame	Frame start + offset	Physical address

Page size 1024. Logical address 2100. Page table says page 2 → frame 7.

Step	Working
Page number	2100 ÷ 1024 = **2** (whole part)
Offset	2100 − 2 × 1024 = **52**
Frame	Page 2 → frame **7**
Physical address	7 × 1024 + 52 = **7220**

The **offset never changes**; only the page number is swapped for a frame number.

Fragmentation in numbers

Situation	Result
Fixed partition of 8 MB holds a 5 MB process	3 MB **internal** fragmentation
Page size 4 KB, process needs 10 KB	3 pages = 12 KB, so 2 KB **internal** fragmentation in the last page
Free holes of 3 MB, 2 MB and 4 MB; a 6 MB process arrives	9 MB free in total but no single hole fits: **external** fragmentation. Compaction would merge the holes.
Buddy system, 1024 KB free, request 100 KB	Split 1024 → 512 → 256 → 128. Allocate a **128 KB** block (smallest power of 2 that fits).

Page replacement trace

Reference string **1, 2, 3, 1, 4, 1, 2** with **3 frames**. F = page fault.

FIFO (remove the page that was loaded first)

Reference	1	2	3	1	4	1	2
Frames after	1	1 2	1 2 3	1 2 3	4 2 3	4 1 3	4 1 2
Fault?	F	F	F	hit	F	F	F

FIFO removes page 1 at the fifth reference because it is the oldest, even though it was just used. **6 faults.**

LRU (remove the page unused for the longest time)

Reference	1	2	3	1	4	1	2
Frames after	1	1 2	1 2 3	1 2 3	1 4 3	1 4 3	1 4 2
Fault?	F	F	F	hit	F	hit	F

At the fifth reference LRU removes page 2 (least recently used) and keeps page 1. **5 faults.**

Optimal (remove the page whose next use is farthest away)

Reference	1	2	3	1	4	1	2
Frames after	1	1 2	1 2 3	1 2 3	1 2 4	1 2 4	1 2 4
Fault?	F	F	F	hit	F	hit	hit

Page 3 is never used again, so Optimal removes it. **4 faults.**

Key idea
Optimal 4 ≤ LRU 5 ≤ FIFO 6 here. Optimal is always the lower bound. LRU usually beats FIFO because of **locality of reference**, but that is a tendency, not a guarantee.

Labs - Code-Reading Cheat Sheet

You are expected to read and interpret the lab code, not rewrite it.

Week 1: a small OS tool (`os`, `platform`)

You see...	You should think...
`platform.system()`, `platform.release()`	Ask the OS for its name and version
`platform.node()`	The machine's network name
`os.listdir(".")`	Ask the **file system** for the entries in the current directory
`open(name, "w")` then `f.write(...)`	Create a file and write to it (uses the `open` system call underneath)
`os.getpid()`	The **PID** of the running process

Week 3: CPU and memory dashboard (`psutil`, `tkinter`, `matplotlib`)

You see...	You should think...
`psutil.cpu_percent()`	Current CPU usage, read from the OS's accounting
`psutil.virtual_memory().percent`	Percentage of memory in use
`deque(maxlen=60)`	Keep only the last 60 readings
`root.after(1000, update)`	Run `update` again in 1 second **without blocking the UI thread**

A `while` loop with `sleep` in the same thread would freeze the window. That is Practice 1 scenario 1.

Week 5: paging simulation

```
physical_memory = [None, 'OS', None, 'AppX', None, None, 'OS', None]
program_pages = ['A', 'B', 'C', 'D']
page_table = {}
available_frames = [i for i, frame in enumerate(physical_memory) if frame is None]

for logical_page_num, page_content in enumerate(program_pages):
    if available_frames:
        physical_frame_num = random.choice(available_frames)
        available_frames.remove(physical_frame_num)
        page_table[logical_page_num] = physical_frame_num
        physical_memory[physical_frame_num] = page_content
```

You see...	You should think...
`physical_memory` (list of 8)	8 **frames**; `None` = free. 3 are occupied (`OS` in 1 and 6, `AppX` in 3), 5 are free: 0, 2, 4, 5, 7
`program_pages`	The program's **logical pages** 0–3
`page_table = {}`	The map **logical page** → **physical frame**
`random.choice(available_frames)`	Any free frame will do: allocation is **non-contiguous**
`available_frames.remove(...)`	Marks the frame as used, so two pages never share a frame
`page_table[logical_address]`	Every access goes **through the page table**

The slide code has two typos: it declares `spage_table = {}` but uses `page_table`, and `logical_address = 2 [ change values here]` is a note, not valid Python.

Week 5: memory-leak detection

tracemalloc.start()	Snapshot 1	Run the suspect code	Snapshot 2	compare_to(snap1, 'lineno')

`tracemalloc` tracks allocations over time. Comparing two snapshots by line number points at the lines that allocated the most new memory.

Paging Assignment - Questions and Answers

These are the team assignment's questions. Be able to reproduce each answer and its reason.

Question	Answer	Why
How many frames, how many occupied, how many free?	8 total; 3 occupied; 5 free (0, 2, 4, 5, 7)	Count the `None` entries
Only one frame free, program loads 4 pages?	Page A is loaded; B, C, D are **silently skipped**; a later access to a missing page raises `KeyError`	`if available_frames:` is false once the list is empty. A real OS would be out of memory and would have to swap pages to disk
`page_table = {0: 4, 1: 0, 2: 7, 3: 2}`, memory `['B','OS','D','AppX','A',None,'OS',None]`: page 3? page 1?	Page 3 → frame 2 → **D**. Page 1 → frame 0 → **B**	Look up the frame, then read that frame
Same example: page 2?	Frame 7, which is printed as `None`	**The assignment's example is inconsistent**: `C` should be in frame 7. If the exam reuses it, follow the page table and say what the frame holds
`logical_address = 5`?	`KeyError: 5`	The program has pages 0–3 only: an **invalid address**
Same page table on every run?	No	`random.choice` picks differently each time
After adding `random.seed(42)`?	The same result every run	A fixed seed makes the random sequence **reproducible**, which matters when testing and debugging
Random frame vs lowest-numbered frame (`available_frames[0]`)?	First-fit packs pages into frames 0, 2, 4, 5 and leaves frame 7 free; random scatters them	Compactness **does not matter** for paging: any page can go in any frame
Does either strategy cause external fragmentation?	**No**	All frames are the same size and pages need not be adjacent
Can two pages land in the same frame?	No	The frame is removed from `available_frames` after use
What if the `remove` line is commented out?	Several pages can get the same frame and **overwrite** each other	Nothing marks the frame as taken
`available_frames[-1]` instead of `random.choice`?	Identical output every run	The choice is deterministic. Randomness only shows that paging works whichever frames are granted
Logical page 4 when frame 4 exists?	Invalid	**Logical page numbers and physical frame numbers are different number spaces.** Frame 4 exists; page 4 does not

Chapter IV - Generalities on Files and Access Methods

What is a file?

A **file** is a named collection of related information recorded on secondary storage. It represents programs, documents, images, videos, or system data.

User / App	File abstraction (names, paths, streams)	File System	Physical disk blocks

Without a file system, secondary storage would be one massive, undifferentiated block of bytes with no way to identify where one piece of data ends and the next begins. The file system provides the structure, naming, indexing, and protection that turn raw sectors into accessible files.

File attributes

A file consists of its data content plus **metadata** (attributes) maintained by the operating system:

Attribute	Plain meaning	Example
Name	Human-readable identifier	`midterm_notes.txt`, `kernel.bin`
Type	Identifies the format or kind of data	`.pdf`, `.py`, `.exe`, `.png`
Location	Device pointer to the physical blocks on disk	Device ID + starting disk block number
Size	Current size in bytes, words, or blocks	14,250 bytes (3 disk blocks)
Protection	Access control rules determining who can read/write/execute	`rwxr-xr--` (Owner: rwx, Group: r-x, Other: r--)
Time / Date	Timestamps for auditing, caching, and backups	Creation, last modification, last access

> **Scope note**
> File attributes are stored in dedicated OS metadata structures (such as **inodes** in Unix/Linux, **File Control Blocks (FCB)**, or the **Master File Table (MFT)** in Windows NTFS), kept separate from the file's raw payload data.

File operations

The operating system provides system calls for six fundamental file operations:

Operation	Action performed by the OS
Creation	Allocates free space on disk and creates a new directory entry with initialized metadata
Writing	Appends or updates data at the current write pointer; advances the pointer
Reading	Retrieves data into memory from the current read pointer; advances the pointer
Deleting	Removes the directory entry and deallocates/releases all data blocks back to free space
Opening / Closing	`open()` loads metadata into the kernel's open-file table; `close()` flushes buffers and frees resources
Seeking	Repositions the internal file pointer to a specified byte offset without reading or writing

File access methods

The access method determines the pattern in which an application reads or writes file records:

Sequential (start to end)	Direct / Random (jump to block n)	Indexed (look up block in index)

Method	How data is accessed	Best suited for	Trade-off
Sequential access	Records are read/written in linear order from beginning to end (`read_next()`). Rewind resets to start.	Compilers, text editors, log processing, media streaming	Simple and fast for streams; cannot jump directly to a record
Direct (Random) access	Any disk block can be accessed immediately by its block number (`read(block_n)`). No need to read preceding data.	Databases (DBMS), transaction systems, virtual machine disk images	Fast arbitrary lookups; requires fixed-size records or block-based addressing
Indexed access	An auxiliary index table maps search keys to disk block addresses. The system searches the index first, then jumps straight to the block.	Large database files with frequent key-based lookups	Very fast retrieval; index consumes additional storage and must be updated on changes

Chapter IV - Directory Structures: Logical and Implementation

Purpose and operations of directories

A **directory** is a special type of file used by the file system to organize, manage, and locate other files and subdirectories.

Core operations: Create a directory, Delete a directory (usually required to be empty), List files, Rename, Traverse (navigate the hierarchy to resolve a path).

Logical directory structures (user perspective)

The logical structure defines how users and applications view the directory tree:

Structure	Description	Advantages	Disadvantages
Single-Level	All files in one single root directory. Simplest model.	Very simple to implement and understand	**Name collisions** (no two files can share a name); linear search is slow; cannot group related files
Two-Level	Master File Directory (MFD) points to separate User File Directories (UFD) for each user.	Prevents name collisions between different users; basic user isolation and privacy	Users cannot share files easily; no subdirectories within a user's UFD; poor scalability for organizing one's own files
Tree-Structured	Hierarchical tree with root directory `/` and arbitrary nested subdirectories.	Modern PC standard (Windows, macOS, Linux); allows deep categorization; easy searching; highly scalable	Users cannot modify the root; path traversal overhead for deep paths; sharing files requires permissions or copies
Acyclic Graph	Extends the tree by allowing files or subdirectories to be shared across multiple directories using links. No cycles/loops allowed.	Allows shared files between multiple users; edits by one user are immediately visible to all	Complex implementation; dangling pointers if target is deleted; requires reference counting (delete file only when all references are gone)
General Graph	Allows directory paths that can loop back to form cycles.	Maximum flexibility; allows any directory linking	Most expensive to implement; search/traversal algorithms can enter infinite loops without cycle detection; **requires garbage collection**

Do not confuse

An **acyclic graph** directory forbids cycles, so deleting a file can be managed with simple **reference counters**. A **general graph** directory permits cycles (directories pointing back to ancestors); reference counting fails with cyclic dependencies, requiring full **garbage collection** to recover abandoned loops.

Directory implementation structures (how it is stored)

The implementation structure defines how the operating system internally stores and searches directory entries:

Linear list (scan from start)	Hash table (compute slot)	Tree-based (traverse B-tree)

1. **Linear List:**

- Sequential array/list of directory entries: `[file1.txt | file2.txt | file3.txt | ... | fileN.txt]`.
- **Search:** linear scan from the beginning. Best: O(1); Worst: O(n); Average: O(n) (checks N/2 entries).
- **Insert:** must perform a full linear search first to prevent duplicate filenames, then append: O(n).
- **Delete:** linear search to locate entry, deallocate space, remove entry from list: O(n).
- **Verdict:** Simple to program, but becomes painfully slow as directories grow. Scales poorly.

2. **Hash Table:**
   - Two parts: an array of slots (buckets) and a **hash function** converting filename string to an array index (`hash(name) % slots`).
   - **Lookup:** compute hash, jump directly to slot: **O(1) average time**.
   - **Collisions:** Two different filenames hash to the same slot (e.g. "Alice" and "Alicia" both hash to slot 0).
     - **Chaining:** each slot holds a small linked list of entries that hashed there.
     - **Open addressing:** if the target slot is taken, probe sequentially for the next empty slot.
   - **Load factor:** $ ext{items} \div ext{slots}$. Low load factor $ ightarrow$ rare collisions $ ightarrow$ fast (close to O(1)). High load factor $ ightarrow$ many collisions $ ightarrow$ slows toward O(n). The table must resize and rehash when the load factor becomes too high.
   - **Verdict:** Fast O(1) average performance, but unordered and requires collision resolution.

3. **Tree-Based (B-Tree, B+ Tree):**
   - Hierarchical balanced trees where entries are kept sorted.
   - **Search, Insert, Delete: O(log n)**.
   - Used in production file systems (NTFS, ext4, APFS, Btrfs) handling millions of files.

Pathname resolution

When an application opens `/home/user/file.txt`, the OS resolves the path step by step: `Root (/) → Traverse "home" → Traverse "user" → Locate "file.txt" in directory entries`

- In a **linear list** of 1,000,000 files: up to **1,000,000 checks** (O(n)).
- In a **balanced B-tree** of 1,000,000 files: only about **20 checks** ($\log_2 1{,}000{,}000 pprox 20$, O(log n)).

Comparison: Linear list vs Hash table vs Balanced tree

Structure	Search time	Insert time	Delete time	Ordered?	Scales well?
Linear list	O(n)	O(n)	O(n)	Yes (insertion order)	❌ No
Hash table	O(1) average	O(1) average	O(1) average	❌ No	✅ Yes
Balanced tree (B-Tree)	O(log n)	O(log n)	O(log n)	✅ Yes (sorted)	✅ Yes

Chapter IV - File Systems Architecture and Types

Primary objectives of a file system

1. **Storage Management:** Efficiently allocate and deallocate disk blocks to prevent fragmentation and maximize storage utilization.
2. **Data Organization:** Structure data hierarchically using directories and subdirectories for intuitive navigation.
3. **Access Control:** Enforce security permissions (read, write, execute) so only authorized users access files.
4. **Data Integrity:** Protect against file corruption and data loss during crashes through journaling and consistency checks.
5. **Performance Optimization:** Minimize slow physical disk I/O through caching, read-ahead, indexing, and smart block layout.

Layered file system structure

Modern operating systems organize file system responsibilities into layers:

User Application	Logical File System	Virtual File System (VFS)	Physical File System	Partitions / Disk Hardware

Layer	Responsibility	Examples
User Application	Software that requests file operations (`open` , `read` , `write` , `delete`)	VS Code, MS Word, Python script
Logical File System	Manages metadata, filenames, directory trees, permissions, and File Control Blocks (FCB / Inodes)	ext4 metadata handler, NTFS directory manager
Virtual File System (VFS)	Kernel middleware that provides a uniform interface/API for multiple file systems	Linux VFS, macOS VFS layer
Physical File System	Handles physical block placement on storage, block translation, and device drivers	Block allocation, sector translation
Partitions	Logical divisions of physical storage where independent file systems reside	Partition 1, Partition 2 (C:, D:, /dev/sda1)

Key distinction

Universal layers: User Application, Logical File System, and Physical File System are present in every OS that supports files. **OS-dependent layers:** The **Virtual File System (VFS)** is kernel middleware in Linux, macOS, and Android that allows multiple file systems (ext4, NTFS, FAT32) to work under one uniform system call interface. Simpler embedded systems omit VFS. **Partitions** are optional storage configurations.

Major file system types

Category	File system	Key characteristics and use cases
Disk-Based	**FAT (File Allocation Table)**	Simple, universal compatibility across USB flash drives and SD cards; uses a central allocation table; no journaling; 4 GB single file limit on FAT32.
	NTFS (New Technology FS)	Modern Windows file system; uses a Master File Table (MFT); supports journaling, file encryption (EFS), ACLs, and massive file sizes.
	ext4 (Fourth Extended FS)	Default Linux file system; supports journaling, extents (ranges of contiguous blocks reducing fragmentation), and high-capacity storage.
	HFS / APFS	Apple file systems; APFS (Apple File System) features 64-bit inodes, crash protection, instant cloning, and built-in full-disk encryption.
Network & Distributed	**NFS (Network File System)**	Allows client machines to access and manipulate files stored on a remote server over a network transparently.
	HDFS (Hadoop Distributed FS)	Distributed, fault-tolerant file system designed for big data storage and map-reduce processing across clusters of commodity hardware.
Special-Purpose	**procfs (`/proc`)**	Virtual pseudo-file system stored entirely in RAM; provides an interface to kernel data structures and running process state (`/proc/cpuinfo` , `/proc/<pid>/status`).
	tmpfs	Fast, volatile file system located in RAM; ideal for temporary build directories and scratch storage; wiped on system reboot.

Chapter IV - Allocation Methods, Free Space and Reliability

Disk allocation methods

Different strategies determine how physical disk blocks are allocated to a file:

Contiguous (consecutive blocks)	Linked (pointers in blocks)	Indexed (all pointers in an index block)

Method	Description	Advantages	Disadvantages
Contiguous Allocation	Files occupy consecutive disk blocks. Directory entry stores: starting block and length.	Fastest sequential and random access (block k is at $ ext{start} + k$).	**External fragmentation**; difficult to expand a file (size must be known in advance or file must be moved).
Linked Allocation	Each disk block contains a pointer to the next block (linked list of blocks). Directory stores pointer to first and last blocks.	**No external fragmentation**; files grow dynamically one block at a time.	**Poor random access** (must traverse the chain from block 0); pointer overhead in each block; poor reliability (one bad pointer breaks the file).
Indexed Allocation	Each file has an **index block** containing an array of pointers to all its data blocks. Directory stores pointer to the index block.	**Fast random access**; no external fragmentation; files can grow dynamically.	**Index block overhead** (even small files consume a full index block); very large files require multi-level indexing (direct, single-indirect, double-indirect).

Free space management

The OS tracks unallocated disk blocks using one of three main methods:

Method	How it works	Trade-off
Bit Vector (Bitmap)	Each block is represented by a single bit: `0 = free`, `1 = allocated`.	Very compact; extremely fast to find the first free block via bitwise CPU instructions; bitmap itself must be kept in memory.
Linked List of Free Blocks	Free disk blocks are linked together; each free block holds a pointer to the next free block.	Simple; zero extra memory overhead outside free blocks; but traversing the list to find multiple contiguous blocks is very slow.
Grouping and Counting	**Grouping:** first free block stores addresses of n free blocks. **Counting:** stores address of first free block and count of contiguous free blocks following it.	Reduces metadata traversal overhead; efficient when blocks are freed in contiguous chunks.

File system protection

- **Access Control:** Enforces permissions: Read (`r`), Write (`w`), Execute (`x`). Grouped into Owner, Group, Others (`chmod 755` = rwxr-xr-x).
- **Security Mechanisms:**
  - **Access Control Lists (ACLs):** Fine-grained per-user and per-group permissions on specific files.
  - **Role-Based Access Control (RBAC):** Permissions assigned based on job role rather than individual user identity.
  - **Encryption at rest:** Encrypts data blocks on disk so stolen physical storage cannot be read.

Reliability and performance techniques

1. **Journaling (Write-Ahead Logging):**

- **Problem:** A system crash or power outage during a file write leaves disk structures in an inconsistent, corrupted state.
- **Solution:** All metadata and block changes are written sequentially to a dedicated circular log (**journal**) *before* applying them to the file system.
- **Recovery:** On reboot, the OS replays the committed journal entries to restore consistency in seconds without running a slow full-disk scan (used in NTFS, ext3/ext4).

2. **Caching and Buffering:**
   - **Buffer cache:** Stores frequently accessed disk blocks in RAM, avoiding slow physical disk seeks.
   - **Read-ahead:** Prefetches subsequent blocks into RAM during sequential file reads.
3. **RAID (Redundant Array of Independent Disks):**
   - **RAID 0 (Striping):** Splits data blocks across multiple disks. Maximizes read/write speed; **zero fault tolerance** (one disk failure destroys all data).
   - **RAID 1 (Mirroring):** Duplicates exact copies of data across two disks. High fault tolerance; cost is 100% capacity overhead (usable capacity = 50%).
   - **RAID 5 (Block Striping with Distributed Parity):** Data and parity blocks are distributed across three or more disks. High performance, survives single disk failure, efficient capacity ($ rac{N-1}{N}$).

Chapter IV - Directory Structure vs File System

The slides highlight a crucial distinction: **The directory structure is the user-friendly interface, while the file system is the underlying engine.**

Aspect	File System	Directory Structure
Definition	The entire framework for storing, managing, and retrieving files on a storage device	The hierarchical layout and organization of folders and files
Scope	Includes block allocation, metadata, inodes, superblock, journaling, access control, free space management	Focuses on how files are named, grouped, organized, and navigated by path
Function	Controls how data bytes are physically stored, retrieved, and protected	Helps users and programs locate and manage files by human-readable names
Example Components	Inode table, superblock, data blocks, FAT, MFT, journal	Root directory `/`, subdirectories like `/home/student/docs`
User Interaction	Mostly behind the scenes (managed by the OS kernel)	Directly visible and navigable by users and applications

Commonly Confused Concepts

Do not confuse	Correct distinction
Program vs process	A file on disk vs that program **in execution** with state.
Process vs thread	Own isolated memory vs shared memory inside a process.
Kernel vs shell	Core that talks to hardware vs interface that takes commands.
System vs application programming	Manages hardware and OS resources vs solves end-user problems.
Monolithic vs microkernel	Services inside the kernel (Linux) vs minimal kernel with services outside (QNX).
Batch vs real-time OS	Jobs run without user interaction vs strict timing constraints.
Multi-user vs distributed OS	Many users on one system vs many computers acting as one system.
Ready vs waiting	Could run now but has no CPU vs cannot run until something (I/O) finishes.
Zombie vs orphan	Terminated, exit status not collected vs still running, parent gone.
Daemon vs background process	A background service that starts at boot vs any process without user interaction.
User-level vs kernel-level thread	Invisible to the OS, one block stalls all vs scheduled by the OS individually.
Concurrency vs parallelism	Taking turns vs running at the same instant on different cores.
CPU-bound vs I/O-bound	Limited by calculation (use multiprocessing) vs limited by waiting (use threading).
Frozen app vs dead process	A blocked thread in a live process vs a process that no longer exists.
Race condition vs crash	A wrong, changing result from unsynchronized access vs the process being killed.
Global vs local variable in threads	Data segment, shared vs the thread's own stack, private.
IPC vs shared memory between threads	Explicit mechanism between processes vs automatic inside one process.
Page vs frame	Block of logical memory vs block of physical memory.
Logical page number vs physical frame number	Index in the program's view vs index in real memory; the page table links them.
Page table vs segment table	Page → frame vs segment → base address and length.
Paging vs segmentation	Fixed-size blocks, no external fragmentation vs variable-size blocks matching program structure, external fragmentation.
Internal vs external fragmentation	Waste **inside** a block vs gaps **between** blocks.
Fixed vs dynamic partitioning	Set at boot, internal fragmentation vs sized on demand, external fragmentation and compaction.
Virtual memory vs RAM	Disk space made to look like main memory vs the real main memory.
Page fault vs error	A normal signal to load a page vs something going wrong.
FIFO vs LRU	Oldest **loaded** vs least recently **used**.
LRU vs Optimal	Looks at the past vs needs the future.
Buddy vs slab	Power-of-2 blocks split and merged vs pre-allocated slabs for kernel objects.
Temporal vs spatial locality	The same data again vs nearby data.

Do not confuse	Correct distinction
File system vs directory structure	Low-level storage engine & framework vs user-facing layout of folders and files.
Sequential vs direct vs indexed access	Read start-to-end vs jump directly by block number vs look up block pointer in index table.
Single-level vs tree-structured directory	Flat namespace with name collisions vs hierarchical tree with subdirectories.
Acyclic graph vs general graph directory	Allows shared files via links with no loops vs allows cycles (which requires garbage collection).
Linear list vs hash table directory	O(n) scan vs O(1) average hash lookup (with collisions).
Contiguous vs linked vs indexed allocation	Consecutive blocks (external fragmentation) vs pointer chain (terrible random access) vs index block of pointers (index overhead).
Internal vs external fragmentation in file systems	Unused space in last block vs scattered unallocatable free blocks between files.
Bitmap vs linked list free space	1 bit per block (compact, fast first-free) vs pointer in each free block (slow traversal).
procfs vs tmpfs	RAM-based view of kernel/processes vs RAM-based volatile storage for fast files.
Journaling vs backup	Crash recovery to consistent disk state via write-ahead log vs external copy of data for disaster recovery.
RAID 0 vs RAID 1 vs RAID 5	Speed only (no redundancy) vs full duplicate mirroring vs distributed parity with 1-drive fault tolerance.
VFS vs physical file system	Kernel abstraction providing unified system-call API vs disk-specific driver handling actual blocks.

How to Answer Scenario Questions

1. "Process or thread problem?"

What is still working?	What does that tell you about address spaces?	Name the mechanism	Give the fix

Model: *The OS still responds, so the process is alive and only its UI thread is blocked. The OS runs in separate address spaces and is scheduled independently. Fix: move the heavy work to a second thread.*

2. "Which design would you choose?"

Need isolation or need speed?	Processes for isolation, threads for speed	State the cost	Offer the hybrid

Model: *Threads are cheaper because they share the address space, but one crash kills them all. Processes isolate faults at the cost of memory and slower switches. A process pool with thread pools gives both.*

3. "CPU-bound or I/O-bound?"

Is the task calculating or waiting?	Calculating: multiprocessing, true parallelism	Waiting: threading, concurrency

4. "Which fragmentation?"

Is the waste inside a block or between blocks?	Inside: internal	Between: external	Name the technique that causes it

5. "Trace the page table"

Find the logical page	Look it up in the page table	Go to that frame	Read the content

If the page is not in the table: the address is invalid (in the lab, `KeyError`).

6. "Compare two techniques"

How is memory divided?	Contiguous or not?	Which fragmentation?	Overhead	Where it is used

Use the comparison table: one row per technique answers the whole question.

7. "Which directory structure or implementation?"

Identify the requirement (sharing? nesting? scale? speed?)	Select the structure	Justify with trade-offs

Model: *A multi-user environment where team members need to share files and see each other's edits. Choice: Acyclic graph directory structure with links. Trade-off: Deletion requires reference counting so shared files are not deleted prematurely.*

8. "Which allocation method fits?"

Sequential or random access? Large or small files? Can size change dynamically?	Contiguous / Linked / Indexed	State the main limitation

Model: *A database requires fast random access and frequent updates of varying record sizes. Choice: Indexed allocation. Contiguous suffers from external fragmentation when files expand, and linked allocation has unacceptable O(n) random traversal time.*

Self-Check Questions

Try to answer without looking back. These are review questions, not copies of the real midterm.

1. Define an operating system.
2. Name the five main roles of an OS.
3. What are the three core components of an OS, and what does each do?
4. Give the description and one example for a batch OS, a real-time OS and a distributed OS.
5. What is the difference between a monolithic kernel and a microkernel?
6. Define system programming and name its four characteristics.
7. What is a system call? Name the three examples from the slides.
8. Give two differences between system programming and application programming.
9. Why is Python used in this course if C is the traditional system language?
10. Define a process. What three things does it consist of?
11. Name the five process attributes.
12. Name the five process states in order.
13. What is the difference between a zombie and an orphan?
14. What is a daemon?
15. Define a thread.
16. Compare user-level and kernel-level threads when one thread blocks.
17. Give four rows of the process vs thread table.
18. When would you choose multiprocessing, and when threading?
19. A program freezes but the rest of the computer works. What is happening?
20. How does a zombie appear, and how do you prevent it?
21. Why can zombies be dangerous if they use no memory?
22. Why does Chrome use a process per tab? Give one advantage and one disadvantage.
23. Why is a thread context switch cheaper than a process context switch?
24. Two threads run `counter++` a million times each and the total is wrong. Why, and what is it called?
25. Whose fault is a race condition, and what tools prevent it?
26. Which memory regions do threads share, and which are private?
27. Name the four goals of memory management.
28. List the memory hierarchy from fastest to slowest.
29. What is locality of reference? Name the two kinds.
30. Internal vs external fragmentation: define each and name a technique that causes it.
31. Fixed vs dynamic partitioning: one difference and the problem of each.
32. What are a page, a frame and a page table?
33. Give two advantages and two disadvantages of paging.
34. How does segmentation differ from paging?
35. Define virtual memory and demand paging. What is a page fault?
36. What does each of FIFO, LRU and Optimal replace?
37. Why is Optimal only a theoretical benchmark?
38. How does the buddy system work, and where is it used?
39. What is slab allocation used for?
40. In the paging lab, with `page_table = {0: 4, 1: 0, 2: 7, 3: 2}` and memory `['B','OS','D','AppX','A',None,'OS',None]`, what does page 3 give? What about page 0? What is odd about page 2?
41. In the paging lab, can two pages end up in one frame? Why or why not?
42. Does paging cause external fragmentation? Explain.

43. Define a file and name its six core attributes.
44. Name the six basic operations that can be performed on files.
45. Compare sequential access, direct access, and indexed access.
46. What are the advantages and disadvantages of a single-level directory structure?
47. How does a two-level directory prevent name collisions, and what is its main limitation?
48. What is the difference between an acyclic graph directory and a general graph directory?
49. Why does general graph directory structure require garbage collection?
50. Compare the time complexity of linear list, hash table, and B-tree directory implementations.
51. What causes collisions in a hash table directory, and how are they resolved?
52. Name the five primary layers in the layered file system architecture. Which are universal?
53. What is the Virtual File System (VFS), and what problem does it solve?
54. Compare contiguous, linked, and indexed disk allocation methods.
55. How does a free-space bitmap work, and why is it efficient?
56. What is journaling (write-ahead logging) in file systems, and what disaster does it prevent?
57. Compare RAID 0, RAID 1, and RAID 5 in terms of performance and fault tolerance.
58. Explain the difference between a directory structure and a file system using the car analogy.

Self-Check Answers

1. Software that manages hardware and software resources and provides common services for programs; an intermediary between users and hardware.
2. Process management, memory management, file system management, device management, user interface.
3. Kernel: the core, interacts directly with hardware. Shell: takes commands and executes them. File system: organizes and manages data storage.
4. Batch: jobs in batches without user interaction (IBM OS). Real-time: strict timing constraints (VxWorks, QNX). Distributed: networked computers managed as a single system (Amoeba, Plan 9).
5. Monolithic keeps most services inside the kernel (Linux); a microkernel keeps the kernel minimal and runs services outside it (QNX).
6. Developing software that provides core services to the hardware and enables application programs to run. Close to hardware; interacts with the OS through system calls; performance-critical; written in low/medium-level languages.
7. A request from a program to the kernel for a service. `fork()`, `exec()`, `open()`.
8. Any two: low-level vs high-level; C/C++/Assembly/Rust vs Python/Java/JavaScript/C#; manages hardware and OS resources vs solves end-user problems; kernels and drivers vs browsers and games.
9. It is high-level, portable and has rich libraries such as `psutil`, so it is good for observing and scripting the OS. C is for building kernels and drivers.
10. An instance of a program in execution: program code (text), current activity (program counter, registers), and resources (memory, files, CPU state).
11. PID, program counter, state, priority, I/O status.
12. New, ready, running, waiting, terminated.
13. A zombie has terminated but still has a process table entry. An orphan is still running but its parent has terminated.
14. A background service that starts at boot.
15. The smallest unit of execution within a process, sharing the process's memory and resources.
16. User-level: the OS cannot see the threads, so one blocking thread can stall the whole program. Kernel-level: the OS schedules each thread, so the others continue.
17. Memory isolated/shared; creation overhead high/low; communication IPC/direct; context switching slower/faster; resources more/less; termination slower/faster.
18. Multiprocessing for CPU-bound work (true parallelism on several cores). Threading for I/O-bound work and responsive UIs.
19. A thread-level failure: the process is alive but its UI thread is blocked. The OS runs in separate address spaces and is scheduled independently.
20. A child terminates and the parent never calls `wait()` / `waitpid()`. Call it after forking, or reap children in a `SIGCHLD` handler.
21. Each one holds a process table entry, and the table is finite. When it fills up, no new process can be created.
22. Isolation and security: each tab has its own address space. Advantage: a crash or exploit stays in one tab. Disadvantage: more memory and more expensive context switches.
23. Threads share the address space, so there is no page-table switch and no TLB flush; only registers, stack pointer and program counter are swapped.
24. `counter++` is load, increment, store, which is not atomic. Two threads can load the same value and one increment is lost. A race condition.
25. The programmer's, for not synchronizing shared data. Mutexes, semaphores, atomics.
26. Shared: code, data (globals), heap, open file descriptors, current working directory, signal handlers. Private: stack, registers, program counter, thread-local storage.
27. Allocation, protection, sharing, optimization.
28. CPU registers, cache (L1/L2/L3), main memory (RAM), secondary storage, tertiary storage, virtual memory.

29. Programs tend to reuse data. Temporal: the same data again. Spatial: data near what was just used.
30. Internal: wasted space inside an allocated block (fixed partitioning, last page in paging). External: unusable gaps between blocks (dynamic partitioning, segmentation).
31. Fixed: partitions set at boot, causes internal fragmentation. Dynamic: blocks sized on demand, causes external fragmentation and needs compaction.
32. Page: fixed-size block of a process's logical memory. Frame: same-size block of physical memory. Page table: maps each page to a frame.
33. Advantages: no external fragmentation; enables virtual memory. Disadvantages: small internal fragmentation in the last page; page-table overhead.
34. Segments are variable-size and match program structure (code, data, stack), with a segment table of base and length. It matches the programmer's view and eases sharing, but causes external fragmentation.
35. Virtual memory: part of secondary storage made to look like main memory. Demand paging: load a page only when it is needed. Page fault: the signal that a needed page is not in memory, so the OS loads it.
36. FIFO: the oldest page. LRU: the least recently used page. Optimal: the page not needed for the longest time.
37. It needs to know future references, which a real system does not have.
38. Memory is kept in power-of-2 blocks. Find the smallest sufficient block; if none, split a larger block into two buddies; on free, merge adjacent free buddies. Linux kernel.
39. Kernel objects: slabs are pre-allocated for frequently used objects, giving fast allocation with no fragmentation for small objects.
40. Page 3 → frame 2 → `D`. Page 0 → frame 4 → `A`. Page 2 → frame 7, but frame 7 is printed as `None`, so page `C` is missing: the assignment's example memory does not fully match its page table.
41. No. The chosen frame is removed from `available_frames`, so it cannot be chosen again. Without that line, pages overwrite each other.
42. No. All frames are the same size and pages do not need to be adjacent, so any free frame can be used.
43. A named collection of related information recorded on secondary storage. Attributes: Name, Type, Location, Size, Protection, Time/Date.
44. Creation, Writing, Reading, Deleting, Opening/Closing, Seeking.
45. Sequential: reads in linear order from beginning to end. Direct: jumps straight to any block by number. Indexed: looks up block pointer in an index table.
46. Advantage: very simple. Disadvantages: name collisions (no two files with same name), slow linear searching, cannot group related files.
47. It separates files into User File Directories (UFDs) pointed to by a Master File Directory (MFD). Limitation: users cannot share files easily and cannot create subdirectories.
48. Acyclic graph allows shared files and directories via links without cycles. General graph allows cycles/loops.
49. Cycles create circular references where reference counters never reach zero, so only garbage collection can identify and reclaim unreachable directory loops.
50. Linear list: O(n) for search, insert, delete. Hash table: O(1) average for search, insert, delete. B-Tree: O(log n) for search, insert, delete.
51. Multiple filenames hash to the same slot/index. Resolved by chaining (linked list in slot) or open addressing (probing for next free slot).
52. User Application, Logical File System, VFS, Physical File System, Partitions. Universal: User Application, Logical File System, Physical File System.
53. Kernel middleware providing a uniform system-call interface for multiple underlying file systems (ext4, NTFS, FAT32).
54. Contiguous: consecutive blocks, fast random access, external fragmentation. Linked: pointer chain, no external fragmentation, poor random access. Indexed: index block of pointers, fast random access, no external fragmentation, index overhead.
55. Each disk block is represented by 1 bit (0 = free, 1 = allocated). Fast to find free blocks using CPU bitwise instructions.

56. Writes metadata changes to a log before committing to disk. Prevents file system corruption during unexpected crashes or power failures.
57. RAID 0: striping for maximum speed, 0 fault tolerance. RAID 1: mirroring for high fault tolerance, 50% storage overhead. RAID 5: distributed parity, survives 1 disk failure, balanced capacity.
58. The directory structure is the user-facing interface (dashboard/steering wheel); the file system is the underlying engine that manages physical storage, blocks, and protection.

Final 15-Minute Revision Checklist

You should be able to...	Check
Define an OS and list its five roles and three core components.	☐
Match the seven OS types with descriptions and examples (correct pairing).	☐
Explain monolithic vs microkernel.	☐
Define system programming, its four characteristics and five key areas.	☐
Reproduce the system vs application programming table and the Python vs C table.	☐
Define a process; list its parts, attributes and five states.	☐
Define the seven process types, especially daemon, zombie and orphan.	☐
Define a thread and the three thread types with examples.	☐
Reproduce the process vs thread table.	☐
Choose multiprocessing or threading for a task and justify it.	☐
Explain the frozen word processor as a blocked thread.	☐
Explain zombies: cause, why the entry stays, fix, and why threads have none.	☐
Explain Chrome's process-per-tab design: advantage, disadvantage, address spaces.	☐
Explain why thread switches are cheap (no page-table switch, no TLB flush).	☐
Explain a race condition with load / increment / store and name the tools that prevent it.	☐
Reproduce the shared vs private memory table for threads.	☐
List the four goals of memory management.	☐
Order the memory hierarchy and explain locality of reference.	☐
Tell internal from external fragmentation and name the technique behind each.	☐
Explain paging, segmentation and segmented paging.	☐
Explain virtual memory, demand paging and page faults.	☐
Trace FIFO, LRU and Optimal on a short reference string.	☐
Explain the buddy system and slab allocation.	☐
Reproduce the comparison-of-techniques table.	☐
Trace the paging simulation and answer the assignment questions.	☐
Read the lab code: `os.getpid()`, `psutil.cpu_percent()`, `root.after()`, `tracemalloc`.	☐
Define a file and list its six attributes (Name, Type, Location, Size, Protection, Time/Date).	☐
Name and explain the six file operations (Create, Write, Read, Delete, Open/Close, Seek).	☐
Compare sequential, direct, and indexed access methods.	☐
Compare directory structures (single-level, two-level, tree, acyclic graph, general graph).	☐
Explain why general graph directories require garbage collection instead of reference counting.	☐
Reproduce the directory implementation comparison (Linear list O(n), Hash table O(1), B-tree O(log n)).	☐

You should be able to...	Check
Trace pathname resolution for `/home/user/file.txt` and compare check counts.	☐
List the five file system layers and distinguish universal layers from VFS and partitions.	☐
Compare disk-based (FAT, NTFS, ext4, APFS), network (NFS, HDFS), and special file systems (procfs, tmpfs).	☐
Compare disk allocation methods (contiguous, linked, indexed) with pros and cons.	☐
Explain free-space bitmap, journaling (write-ahead logging), and RAID 0 / 1 / 5.	☐
Explain directory structure vs file system using the interface vs engine distinction.	☐

Best final strategy

Almost every answer in this course reduces to one sentence: **processes have separate address spaces, threads share one, and the page table is what makes the separation real.** Practise explaining each Practice 1 scenario with that sentence.

End of Midterm Study Guide